

# Computer Vision Lecture 1a.

Research area of the lecturer:  
3D Human Sensing

Dimitrios Dzionas

Emre Ullrich [emre.ullrich@uva.nl](mailto:emre.ullrich@uva.nl) (tuesday present)

TA: [p.j.adema@uva.nl](mailto:p.j.adema@uva.nl)  
Peter Adema.

labs: at least 20 LC mandatory for grading

WC: practice / discussion.  
work on a part exam.

Deadline: wednesday 01/04/2026 23:59  
form groups (probably with Thomas)

- ADD team's names for TA
- Answers must be separate per question  
Ideally in LaTeX

The Lab 1: Incorporation @ Histograms  
is problems 1a, 1b, 1c, 1d, 1e,  
2,  
3a, 3b, 3c, 3d,  
4a, 4b

submission deadline for both coding  
40 theory assignments: Sunday 23:59

I mark exam will be assessed  
mark exam assessed in two parts

pt. 1 . before lecture-free week

pt. 2 . last week

HL workshops: bonus of 0.5

# Images and Interpolation

concepts:

- Functions
  - Cont. / Disc. space
  - Sampling grid
  - Quantization
  - Interpolation
  - Histograms.
- 

Pixels:

'Atoms' of an image

Pixels are not rectangular elements.

each pixel  $\rightarrow$  gray 'luminance' value  
from image to puzzle.

Physical observation

$\downarrow$  mathematical  
model

Image

light hits object & bounces  
to the sensor

measure electromagnetic energy

@ every point of  
• eye's retina

• camera sensor

Camera will be discussed in  
week 5

function over a spatial domain  
(position on sensor)

sensor as 2D plane  $E \in \mathbb{R}^2$   
points on 2D plane  $x \in E$

Image  $\rightarrow f(x)$

value  $f$  proportional to measured  
energy at position  $x$

defined over a continuous spatial  
domain

## Measuring light

how to measure light over a continuous domain.

In theory  $\rightarrow$  infinite points  $x \in E$

in practice  $\rightarrow$  sampling probes of finite:  
Number sample locations  
Size & sampling areas  
temporal averaging

Integrate sensor energy

$\rightarrow$  over finite area  $\mathcal{R}$  for finite time

choose sampling grid's distance and area size.

if the area size is much greater

$\rightarrow$  for smaller area:  $E+$  will average everything that falls in the small area

$\rightarrow$  for bigger area: a lot of information falls inside the sampling probe  
the sampling probe will average, so averaging means losing information or the image will look more blurry (you lose high frequency details), but with smaller size probes, you preserve details. overlap  $\rightarrow$  more blurry

the underlying mathematical model  
of one part of an image would be  
(height = luminance)  
(continuous)

the vertical value correspond to each  
gray-scale value.

So the image pixels and the model  
encode the same information

What computers perceive as image  
is called a discrete representation

### Definition

Image  $f$  maps from a spatial domain  $E$   
to a range  $V$

from element  $x \in E$   
to value  $f(x) \in V$

$$f : E \rightarrow V$$

→ Euclidian space  $E \in \mathbb{R}^n$

$E \in \mathbb{R}^2$  for 2D photos

$E \in \mathbb{R}^3$  for a scan.

$$E \in \mathbb{R}^2:$$

the basis can be arbitrary  
the coordinate frame can change,  
but signal stays the same.

Range  $V$

at each element  $x \in E$   
we measure ....

gray scale images  $f(x) \in \mathbb{R}$

color images

$$f(x) = \begin{bmatrix} b(x) \\ g(x) \\ r(x) \end{bmatrix} \in \mathbb{R}^3$$

Indicator images:

Binary mask / segmentation

$$\{f(x) \in \{0, 1\}\} \quad f(x) = \begin{cases} 0 \rightarrow \text{background} \\ 1 \rightarrow \text{person} \end{cases}$$

with more: semantically segmentation.

## Discretization:

How can we store a continuous grayscale image:

$$f: \mathbb{R}^2 \longrightarrow \mathbb{R}^1$$

mapped to grayscale value

Discretize:

$\left\{ \begin{array}{l} \text{Spatial} \rightarrow \text{Sampling} \\ \text{Image range} \rightarrow \text{Quantization} \end{array} \right.$

Quantize range:

per pixel: 1 byte / 8 bit / 256 values  
insufficient to store over the values

range:  $0 \dots 255$

$2^8 = 256$  values, is what we  
can store

values  $> 255 \rightarrow$  bit overflow  
values  $< 0$

cannot be encoded.



3 byte / 24 bit

8 bit  $\rightarrow$  red

8 bit  $\rightarrow$  green

8 bit  $\rightarrow$  blue



opengl can switch

for opengl: BGR

tensor for color image:

height  $\times$  width  $\times$  3

for all processing switch to float  
quantize to uint only at  
the end to visualize

float allows you to store all numbers during  
manipulation, but more computationally  
expensive (memory). So that's why you switch  
back at the end.

$f = \text{image}$

$f.\text{max}(\text{min})$ : max or min value represented in  
the image

$g = (f - f.\text{min}()) / (f.\text{max}() - f.\text{min}()) * 255$

$h = 255 * ((f - f.\text{min}()) / (f.\text{max}() - f.\text{min}()))$

$g$  and  $h$  look the same,  
 but with  $g$  you get the normal  
 image in floats back,  
 but with  $h$  you get the inverse  
 because left-to-right in code,  
 changes importance, order of execution

$$g = \underbrace{\left( \frac{f - f_{min}}{f_{max} - f_{min}} \right)}_{\text{normalize}} * \underbrace{255}_{\text{multiply}}$$

$$h = \underbrace{(255 (f - f_{min}))}_{\text{multiply}} \underbrace{\left( \frac{1}{f_{max} - f_{min}} \right)}_{\text{normalize}}$$

problem all the intermediates are  
 still unsigned integers (uint8)  
 so, we get big values we can not  
 represent in a single byte.

So if you do the math for something like:

$255 * (t - f_{min})$  it would be for example

$$f = 50$$

$$f_{min} = 20$$

$$f_{max} = 200$$

7650, but python would give 226, because one byte (uint 8) cannot encode numbers 10 or 255  
→ overflow!

sample domain:

Image pixels  $\rightarrow$  apply rect.  
sampling grid  
over continuous function  
 $f$

continuous images  $\rightarrow f: \mathbb{R}^2 \rightarrow \mathbb{R}_+$

discrete images:  $F: \mathbb{Z} \rightarrow \mathbb{R}$

origin is always top-left  
indices  $i$  and  $j$

$$F(i, j) = f(i \Delta x, j \Delta x)$$

$i, j \in \mathbb{Z}$

usually  $\Delta x = \Delta y = 1$

$i$  = row  
 $j$  = column

same a bigger area)

Week 3: local operators

discretization:

how to go from real image (continuous  $f$ )  
to discrete image ( $F$ )

Interpolation:

from discrete to continuous

→ loses information

What doesn't fall into the sampling  
probes, we lose that information

We will do informed guessing to do  
this

given samples  $F$  of continuous  $f$

Interpolation → calculate value  
 $f(x, y)$

between samples  $F(i, j)$

Assuming: Well-sampled image

Information theory:

Nyquist - Shannon sampling theorem

$F(x), x \in \mathbb{Z} \rightarrow$  input samples of  $f$

$$f(x) = \sin(x), x \in \mathbb{R}$$

$\rightarrow$  continuous 'target' function  
that we want to approximate  
goal: guessimate  $\hat{f}$  that approximates  
 $f$  (input) samples  $F$

$$\hat{f}(x) = F(\lfloor x + 0.5 \rfloor) \rightarrow \text{copy 'floor function' uses sample}$$

$\text{round}(x) \rightarrow \text{python}$

$\lceil x \rceil \rightarrow \text{largest int} \leq x$

$$\begin{aligned}\hat{f}(0.1) &= F(\lfloor 0.1 + 0.5 \rfloor) \\ &= F(\lfloor 0.6 \rfloor) \\ &= F(0)\end{aligned}$$

$$\begin{aligned}\hat{f}(0.5) &= F(\lfloor 0.5 + 0.5 \rfloor) \\ &= F(\lfloor 1.0 \rfloor) \\ &= F(1)\end{aligned}$$

$$\begin{aligned}\hat{f}(1.2) &= F(\lfloor 1.2 + 0.5 \rfloor) \\ &= F(1)\end{aligned}$$

$$\begin{aligned}\hat{f}(3.7) &= F(\lfloor 3.7 + 0.5 \rfloor) \\ &= F(\lfloor 4.2 \rfloor) \\ &= F(4)\end{aligned}$$